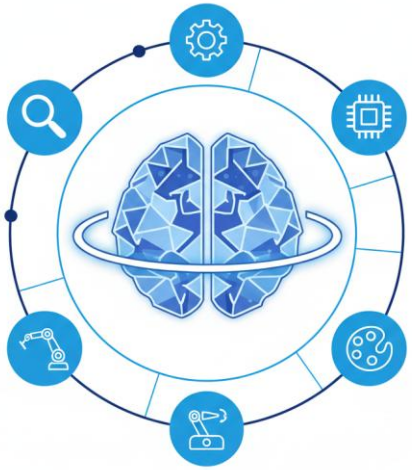
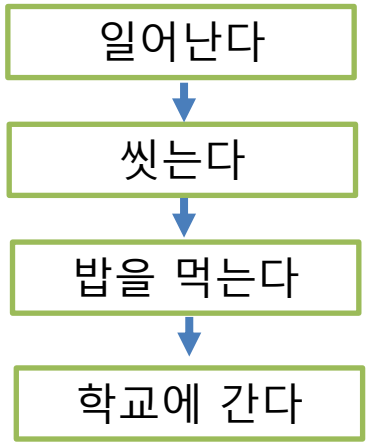
파이썬 객체와 모듈

함수/ 클래스/ 모듈

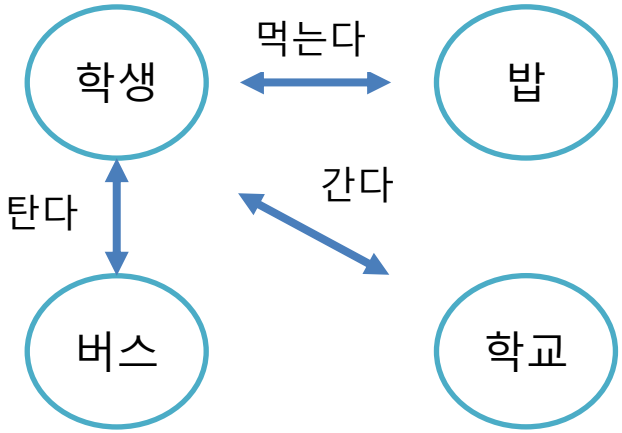


클래스와 객체

- **객체(Object)란?**
  - "의사나 행위가 미치는 대상" -> 사전적 의미
  - 구체적, 추상적 데이터 단위 (구체적- 책상, 추상적-회사)
- **객체지향 프로그래밍(Objected Oriented Programming)**
  - 객체를 기반으로 하는 프로그래밍
  - 먼저 객체를 만들고 객체 사이에 일어나는 일을 구현함.



<절차지향 –C언어>



<객체지향 – Python>

클래스와 객체

- **클래스란?**
  - 객체 지향 프로그래밍(OOP)의 기본 개념 중 하나로, 관련된 속성과 메서드를 하나의 단위로 묶는 틀입니다.
  - 클래스를 사용하면 코드의 재사용성과 유지보수성을 높일 수 있습니다.
  - 클래스는 설계도와 같으며, 이를 기반으로 여러 객체(인스턴스)를 생성할 수 있습니다.
- **클래스의 속성과 기능**
  - 객체의 특성(property), 속성(attribute) -> **멤버 변수**
  - 객체가 하는 기능 -> **멤버 함수**

클래스와 객체

- 클래스 정의 및 사용

```
class 클래스 이름 :
    def __init__(self):
        멤버변수

    def 함수이름(self):
        return
```

```
객체(인스턴스) = 클래스( )

객체.멤버변수
객체.멤버 메서드
```

객체를 생성한 후 점(.) 연산자를 사용하여 멤버변수나 메서드에 접근함

클래스와 객체

- **생성자(constructor)**
  - 객체가 생성될때 자동으로 호출되는 특별한 메서드
  - 생성자는 **__init__(매개변수)**의 형태로 작성하고, 리턴값이 없다
  - 클래스 내의 모든 함수(메서드)의 매개변수에 **self**를 넣어줌

Bike	
color gears	속성(Attributes)
ride() get_info()	메서드(Methord)

클래스와 객체

- **Bike 클래스 정의 및 사용**

```python
# 클래스의 생성자(Constructor)
class Bike:
    # 생성자 메서드 - 객체가 생성될 때 자동으로 호출되는 특별한 메서드
    def __init__(self, color, gears):
        self.color = color  # 자전거 색상
        self.gears = gears  # 자전거 기어 수

    def ride(self):
        print(f"{self.color} 자전거가 {self.gears}단 기어로 달린다.")

    def get_info(self):
        return f"자전거 색상: {self.color}, 기어 수: {self.gears}"

# 클래스 사용 예시
if __name__ == "__main__":
    my_bike = Bike("blue", 21)
    print(my_bike.get_info())
    my_bike.ride()
```

모듈(Module)과 패키지

- **모듈(module)**

  모듈은 Python 파일 하나를 의미하며, 다른 Python 프로그램에서 모듈의 함수, 클래스 등을 재사용할 수 있다.

  - **사용자 모듈 정의**

```python
# my_lib/todos.py
# 요리하다
def cook(name):
    return f"{name} 요리를 합니다."

# 청소하다
def clean(place):
    return f"{place}를 청소합니다."
```

모듈(Module)과 패키지

- **주요 모듈 및 패키지**

모 듈	설 명
math	수학 계산과 관련된 모듈
datetime	날짜 및 시간과 관련된 모듈
time	시간과 관련된 모듈
random	난수를 발생시키는 모듈
os	운영 체제(OS) 자원 제어 관련 모듈

math 모듈

- **math** 모듈

기능	함수의 사용
올림	**math.ceil(2.54)** # 3
내림	**math.floor(2.54)** # 2
제곱근	**math.sqrt(16)** # 4.0
원주율	**math.pi** # 3.1415

math 모듈

- **math 모듈**

```python
import math

# 올림
print(math.ceil(2.54))

# 반올림 - math 모듈이 아님
print(round(2.54))

# 내림(버림)
print(math.floor(2.54))

# 제곱근 - 실수로 반환
print(math.sqrt(2))
print(math.sqrt(25))
```

```
3
3
2
1.4142135623730951
5.0
3.141592653589793
원의 넓이: 50.27
```

datetime 모듈

- **datetime 모듈**

```python
import datetime

# datetime.datetime - 날짜와 시간을 사용
now = datetime.datetime.today() #오늘 날짜
print(now) # 2025-05-18 11:16:33.750807
# 년, 월, 일 출력
print(now.year)
print(now.month)
print(now.day)

# 현재 날짜 표기
print(f"{now.year}. {now.month}. {now.day}.")

# 시, 분, 초 출력
print(now.hour)
print(now.minute)
print(now.second)

# 현재 시간 표기
print(f"{now.hour} : {now.minute} : {now.second}")
```

```
2025-08-01 10:10:24.530677
2025
8
1
2025. 8. 1.
10
10
24
10 : 10 : 24
2025-08-15
2025-08-01
```

datetime 모듈

- **datetime 모듈**

```
# 특정한 날짜 설정
# datetime.date - 날짜만 사용 가능
the_day = datetime.date(2025, 8, 15)
print(the_day)

today = datetime.date.today()
print(today)

# DDay 계산
print("광복절까지 몇일 남았나(DDay)?")

remain_day = the_day - today
print(f"광복절까지 {remain_day.days}일 남았습니다.")
```

```
2025-08-15
2025-08-01
광복절까지 몇일 남았나(DDay)?
광복절까지 14일 남았습니다.
```

Random 모듈

+ 파이썬의 random 모듈을 사용하는 방법에 대해서 알려줘

1 random 모듈 불러오기

python 코드 복사

```python
import random
```

---

2 기본 난수 생성

🔹 0 이상 1 미만의 실수

python 코드 복사

```python
random.random()
```

👉 예: `0.3748293`

🔹 특정 범위의 정수

python 코드 복사

```python
random.randint(1, 10)
```

👉 **1 이상 10 이하** 정수 하나

Random 모듈

- **random 모듈**

```python
# 난수 시드 설정 - 동일한 시드를 사용하면 동일한 난수 시퀀스 생성
random.seed(42)
print(f"시드 설정 후 난수: {random.randint(1, 10)}")
random.seed(42)
print(f"시드 설정 후 난수: {random.randint(1, 10)}")
random.seed()  # 시드 초기화
print(f"시드 초기화 후 난수: {random.randint(1, 10)}")

# 동전 던지기 시뮬레이션
# '앞면' 또는 '뒷면' 중 하나를 무작위로 선택
# 방법1
coin = random.randint(0, 1)
if coin == 0:
    print("동전 던지기 결과: 앞면")
else:
    print("동전 던지기 결과: 뒷면")

# 방법2
coin = random.choice(['앞면', '뒷면'])
print(f"동전 던지기 결과: {coin}")
```

Random 모듈

```python
# 주사위 굴리기 시뮬레이션
dice_roll = random.randint(1, 6)
print(f"주사위 굴리기 결과: {dice_roll}")

# 로또 번호 생성 (1부터 45 사이의 숫자 중 6개 선택)
lotto_numbers = random.sample(range(1, 46), k=6)
print(f"생성된 로또 번호: {sorted(lotto_numbers)}")

# 리스트에서 무작위로 요소 선택
fruits = ['사과', '바나나', '딸기', '감', '사과']
selected_fruit = random.choice(fruits)
print(f"선택된 과일: {selected_fruit}")

# 리스트에서 무작위로 여러 요소 선택 (중복 허용)
selected_fruits = random.choices(fruits, k=2)
print(f"선택된 과일들 (중복 허용): {selected_fruits}")

# 리스트에서 무작위로 여러 요소 선택 (중복 비허용)
selected_fruits = random.sample(fruits, k=2)
print(f"선택된 과일들 (중복 비허용): {selected_fruits}")
```

Random 모듈

- 숫자 맞추기 게임
  - 챗GPT 프롬프트 입력

파이썬으로 숫자 맞추기 게임을 만들 거야.
1~100까지 무작위 숫자를 생성하고,
내가 숫자를 입력하면 무작위 숫자랑 비교해서 숫자가 큰지 작은지 알려줘
숫자를 맞추면 몇 회 만에 맞췄는지 알려주고 게임을 종료해

Random 모듈

- 숫자를 추측해서 맞추는 게임

```python
import random

# 1~100 사이의 무작위 숫자 생성
answer = random.randint(1, 100)
# print(answer) # 디버깅용 출력, 실제 게임에서는 제거할 것

count = 0  # 시도 횟수

print("숫자 맞추기 게임!")
print("1부터 100 사이의 숫자를 맞춰보세요.")

while True:
    guess = int(input("숫자를 입력하세요: "))
    count += 1

    if guess > answer:
        print("더 작은 숫자입니다.")
    elif guess < answer:
        print("더 큰 숫자입니다.")
    else:
        print(f"정답입니다! {count}회 만에 맞췄어요.")
        break
```

AI 사용 – 코파일럿(copilot)

❖ 생성형 AI 코파일럿

▪ 챗GPT 프롬프트 입력

+ ai 코파일럿에 대해 설명해 줘

좋은 질문이에요 👍
AI 코파일럿(AI Copilot)은 말 그대로 *사람을 대신하는 AI*가 아니라, **사람 옆에서 함께 일해주는 똑똑한 조수**라고 보면 딱 맞아요.

🤖 AI 코파일럿이란?

AI 코파일럿은 **인공지능이 사용자의 작업을 보조**해 주는 시스템이에요.
사용자의 **의도·맥락·데이터**를 이해해서 제안, 자동완성, 분석, 요약 같은 걸 해줍니다.

> ✈️ 비유하자면
> 비행기 조종사 = 사람
> 코파일럿 = AI (옆에서 체크하고, 추천하고, 자동 처리)

AI 사용 – 코파일럿(copilot)

❖ 생성형 AI 코파일럿

▪ 챗GPT 프롬프트 입력

+ GitHub Copilot을 사용하는 방법을 알려줘

✔ **GitHub 계정**

- https://github.com ↗
- 로그인만 하면 OK

✔ **지원되는 에디터**

가장 많이 쓰는 건:

- **VS Code** ⭐ (추천)
- JetBrains (PyCharm, IntelliJ 등)
- Visual Studio

👉 여기서는 **VS Code 기준**으로 설명할게요.

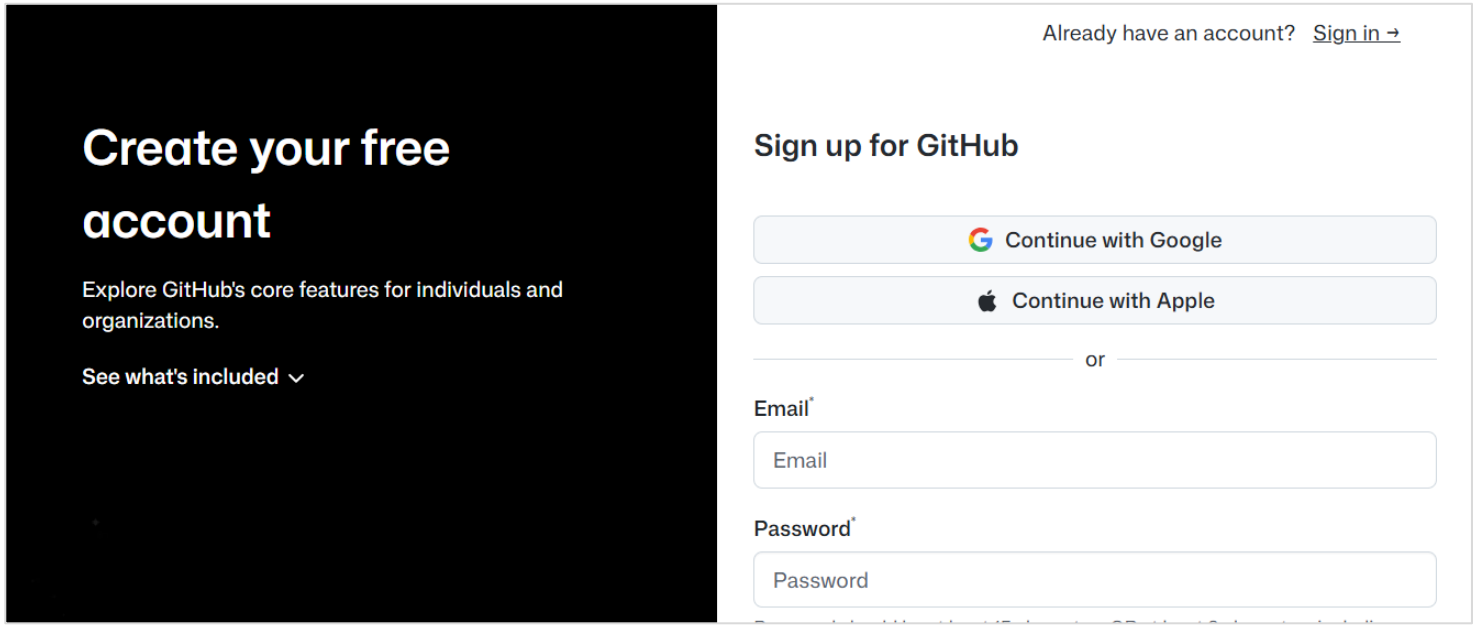
✔ **GitHub Copilot 구독**

- 개인: 유료 (무료 체험 있음)
- 학생/교사: 무료
- 기업: 별도 플랜

AI 사용 – 코파일럿(copilot)

❖ 깃허브 계정 만들기

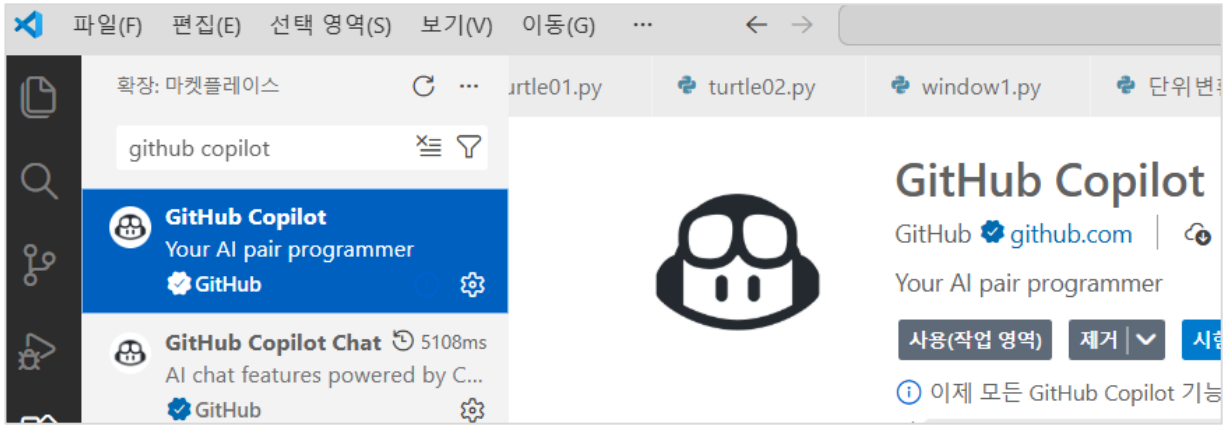
https://github.com/ > 회원가입(sign up)



AI 사용 – 코파일럿(copilot)

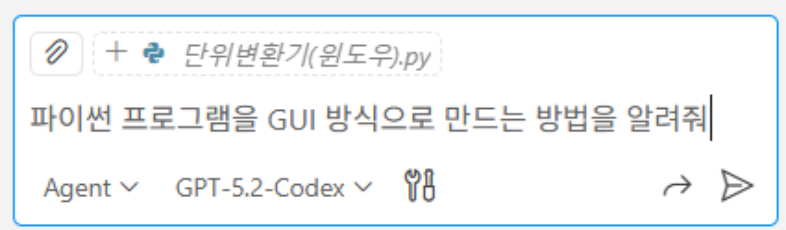
❖ VS Code에서 GitHub Copilot 사용하기

확장 > github copilot 검색 > 설치



윈도우 프로그래밍

- **코파일럿 프롬프트 입력**



- **GUI 라이브러리 선택**: 표준 `tkinter`(가장 간단), `PyQt/PySide`(기능 풍부), `wxPython` 등
- **기본 흐름**: `창 생성 → 위젯 배치 → 이벤트 연결 → 실행`

예) `tkinter` 최소 구조

- `Tk()`로 창 생성
- `Label`, `Entry`, `Button` 등 위젯 추가
- `command`로 버튼 클릭 이벤트 연결
- `mainloop()`로 실행

윈도우 프로그래밍

- **GUI(Graphical User Interface)란?**

  그래픽 사용자 인터페이스를 줄여서 GUI라고 한다. GUI는 '화면'에 표시된 메뉴나 버튼으로 사용자와 상호 작용을 하는 간단한 프로그램이다.

  tkinter 라이브러리를 사용한다. -> **import tkinter**

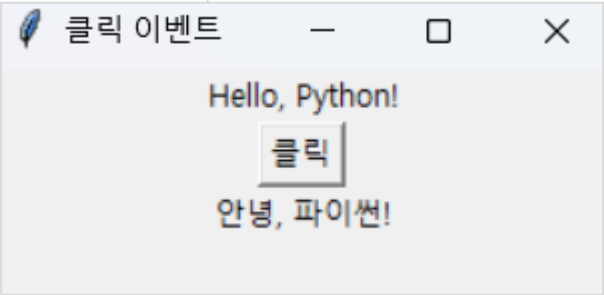
1. Tk 루트

개체이름	클래스
루트	Tk()
프레임	Frame
레이블	Label
입력상자	Entry
버튼	Button
출력상자	Text

윈도우 프로그래밍

- 클릭 이벤트 – 버튼을 눌렀을때 문자 출력

```python
from tkinter import *

# 버튼 클릭시 호출되는 함수
def click():
    output.config(text="안녕, 파이썬!")

root = Tk() # 윈도우 창 생성
root.title("클릭 이벤트") # 창 제목 설정
root.geometry("250x100+200+100") # 가로x세로+x좌표+y좌표

# 라벨과 버튼 추가
# pack(): 위젯을 창에 추가하는 메서드
Label(root, text="Hello, Python!").pack()
Button(root, text="클릭", command=click).pack()

# 클릭후 출력 라벨
output = Label(root, text="")
output.pack()

# 이벤트 루프 시작
root.mainloop()
```



윈도우 프로그래밍

- **Button(버튼) – command**

Button(frame, text="확인", command=**click**).pack()

※ click에 괄호를 하면 함수 생성시점에서 작동하고, 괄호를 생략하면 클릭이 발생한 때 작동함

1. 콘솔에 출력

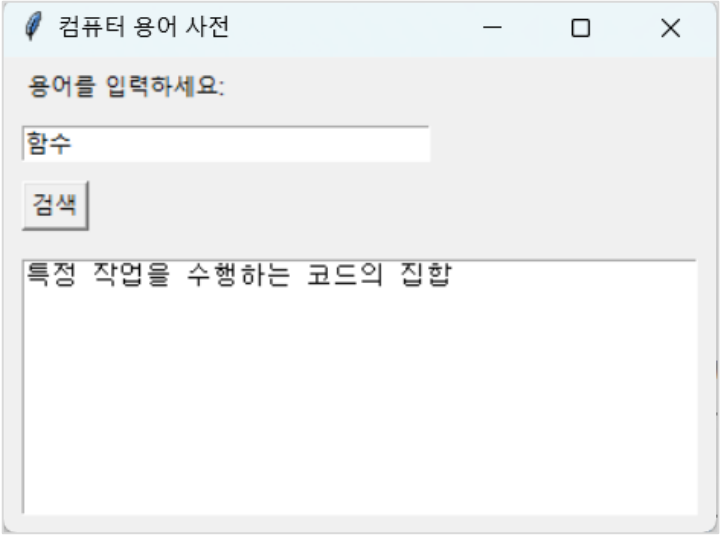
```
def click():
    print("안녕~ 파이썬!")
```

2. 레이블에 출력

```
def click():
    output.config(text="안녕, 파이썬!")
```

컴퓨터 용어사전 앱

- **컴퓨터 용어 사전 – dictionary 자료구조 이용**



App 설명

- 용어를 미리 정의한다. – 딕셔너리 자료 구조
- 단어를 입력하고 검색 버튼을 누르면, 텍스트 상자에 뜻이 출력된다.
- 검색된 단어가 없는 경우 '사전에 없는 용어입니다..'고 출력된다.

컴퓨터 용어사전 앱

- 컴퓨터 용어사전.py

```
from tkinter import *
dict = {
  "알고리즘": "문제를 해결하기 위한 절차나 방법",
  "함수": "특정 작업을 수행하는 코드의 집합",
  "변수": "데이터를 저장하는 메모리 공간의 이름",
  "클래스": "객체 지향 프로그래밍에서 객체를 생성하기 위한 틀",
}

def search():
  # 입력된 단어로 사전 검색
  try:
    word = entry.get().strip() # 입력창에서 단어 가져오기
    if not word: # 단어가 비어있으면
      output.insert(END, "용어를 입력하세요.")
      return
    meaning = dict[word] # 사전에서 뜻 찾기
    output.delete(1.0, END) # 1번째 줄, 0번째 문자 = 첫 번째 위치
    output.insert(END, meaning) # 끝 위치 (텍스트의 마지막)
  except KeyError:
    output.delete(1.0, END)
    output.insert(END, "사전에 없는 용어입니다.")
```

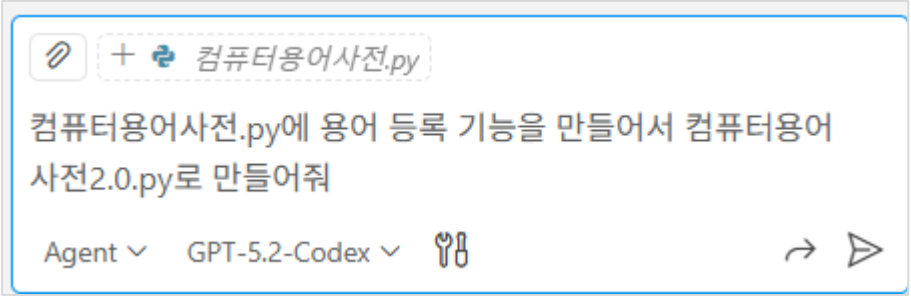
컴퓨터 용어사전 앱

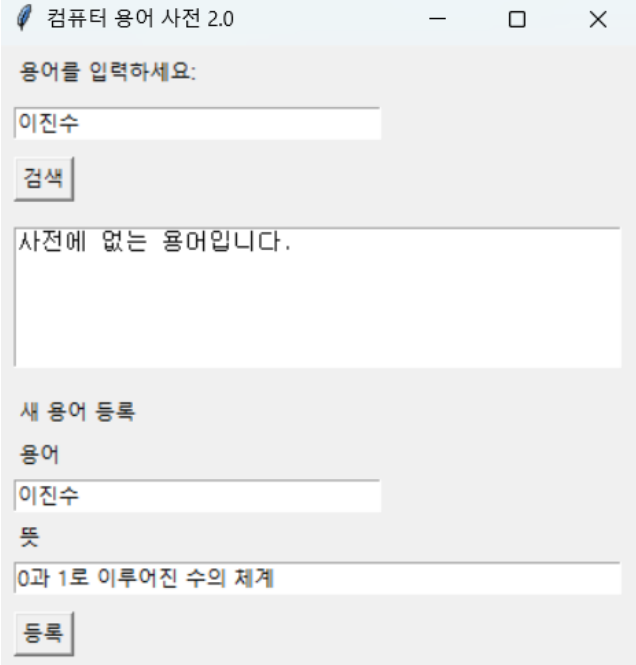
- 컴퓨터 용어사전.py

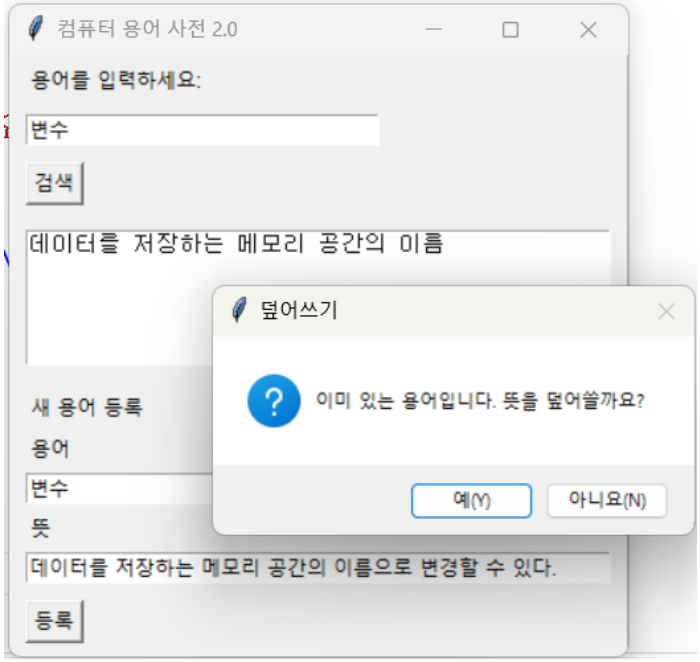
```
root = Tk()
root.title("컴퓨터 용어 사전")

Label(root, text="용어를 입력하세요:") \
    .grid(row=0, column=0, sticky=W, padx=10, pady=5) # 라벨 추가

entry = Entry(root, width=30)
entry.grid(row=1, column=0, sticky=W, padx=10, pady=5) # 입력창 추가

Button(root, text="검색", command=search) \
    .grid(row=2, column=0, sticky=W, padx=10, pady=5) # 버튼 추가

output = Text(root, width=50, height=10)
output.grid(row=3, column=0, sticky=W, padx=10, pady=10) # 출력창 추가

root.mainloop()
```

컴퓨터 용어사전 앱

- **코파일럿 채팅 입력**

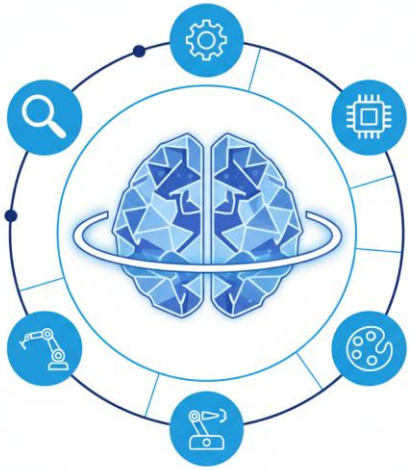






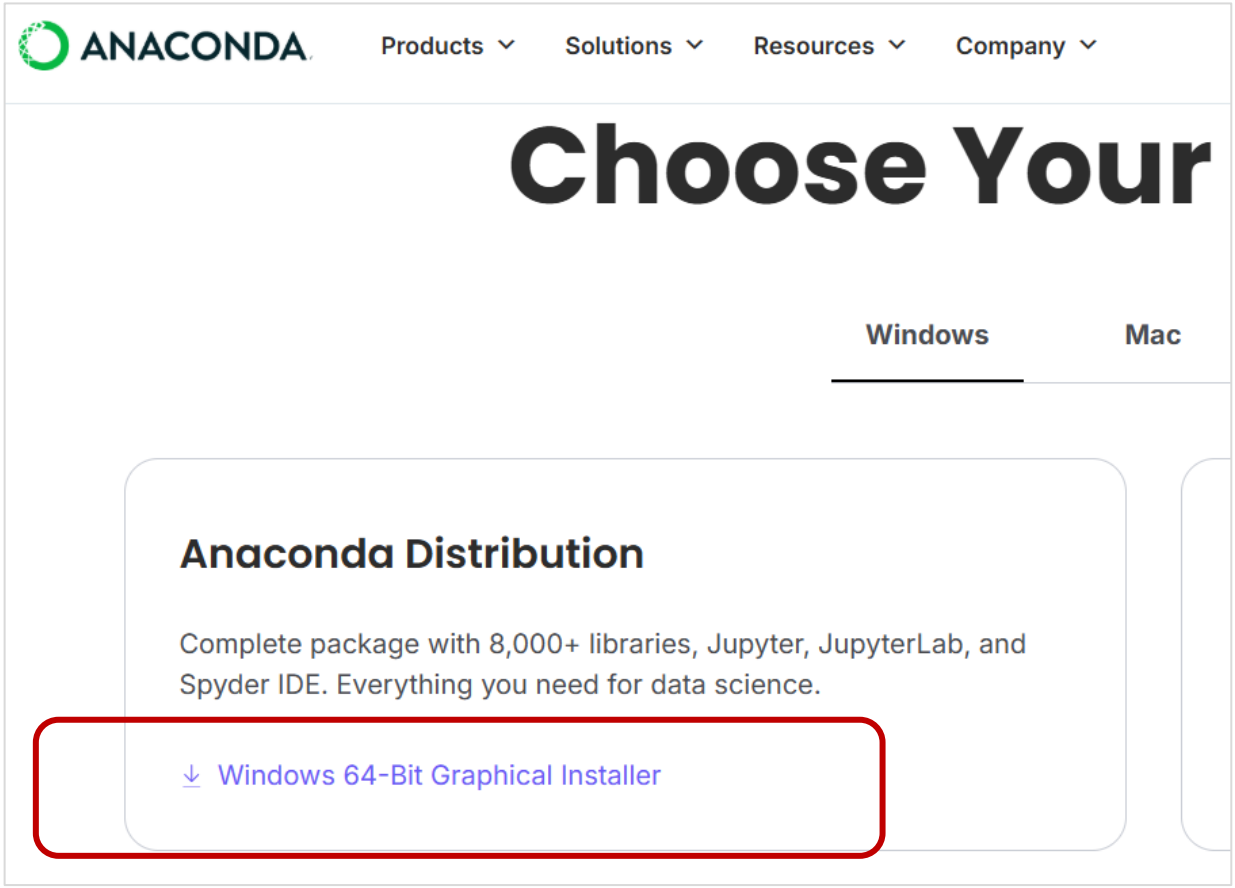
데이터 분석 및 시각화

pandas / matplotlib



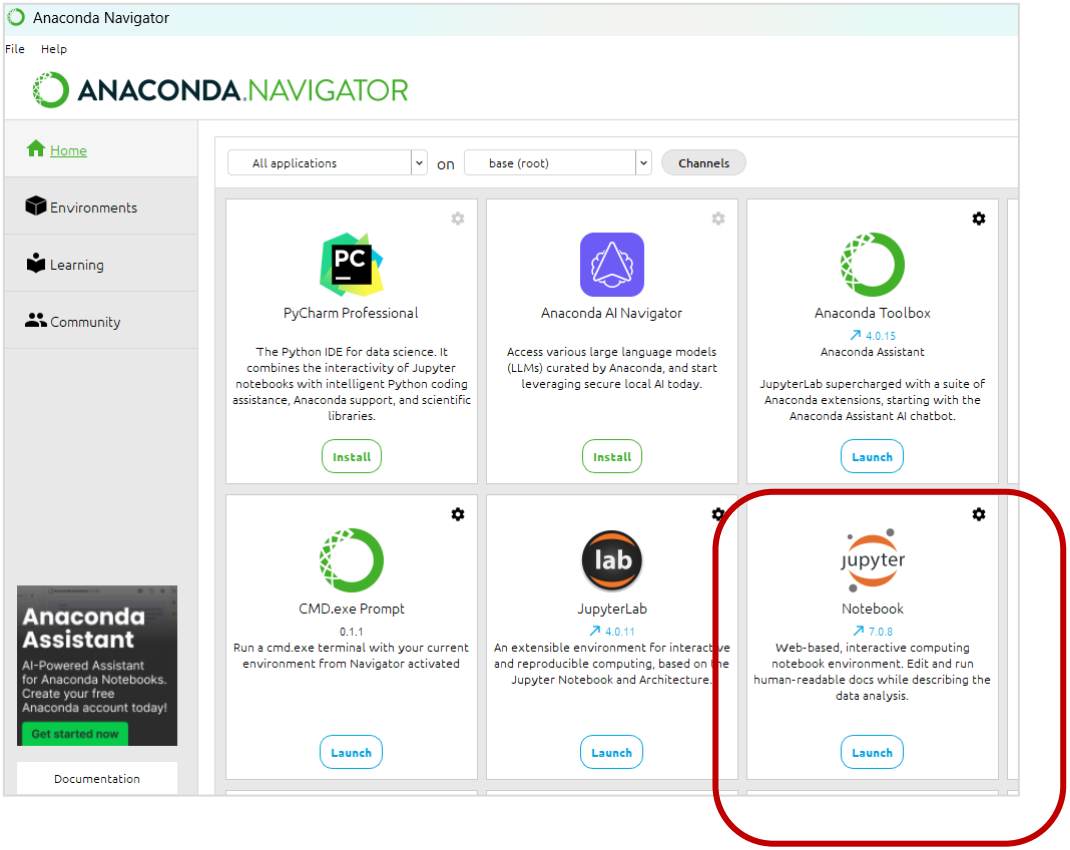
아나콘다(Anaconda) 플랫폼

❖ **Anaconda Download**



아나콘다(Anaconda) 플랫폼

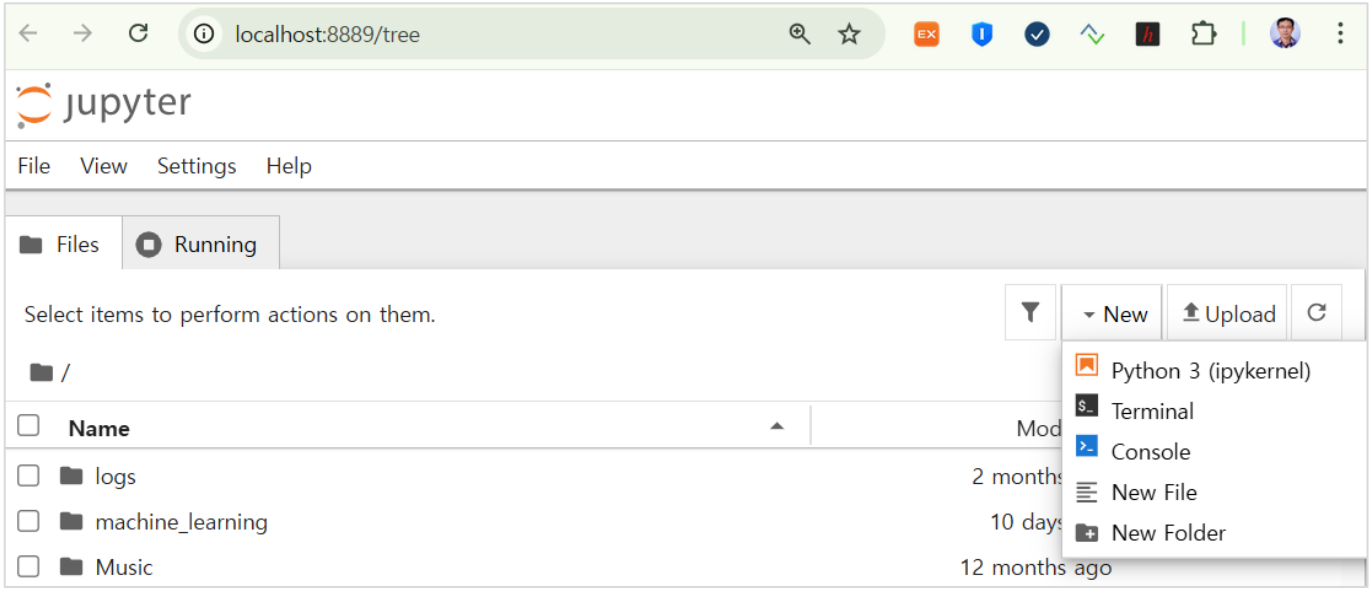
❖ 주피터 노트북(Jupyter Notebook) IDE 실행

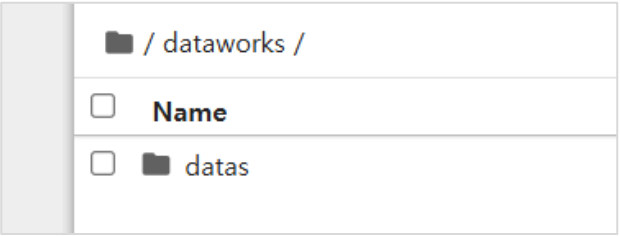


주피터 노트북(Jupyter Notebook)

❖ 작업 디렉터리 생성

New > New Folder > dataworks 폴더 생성 > 하위에 datas 폴더 생성

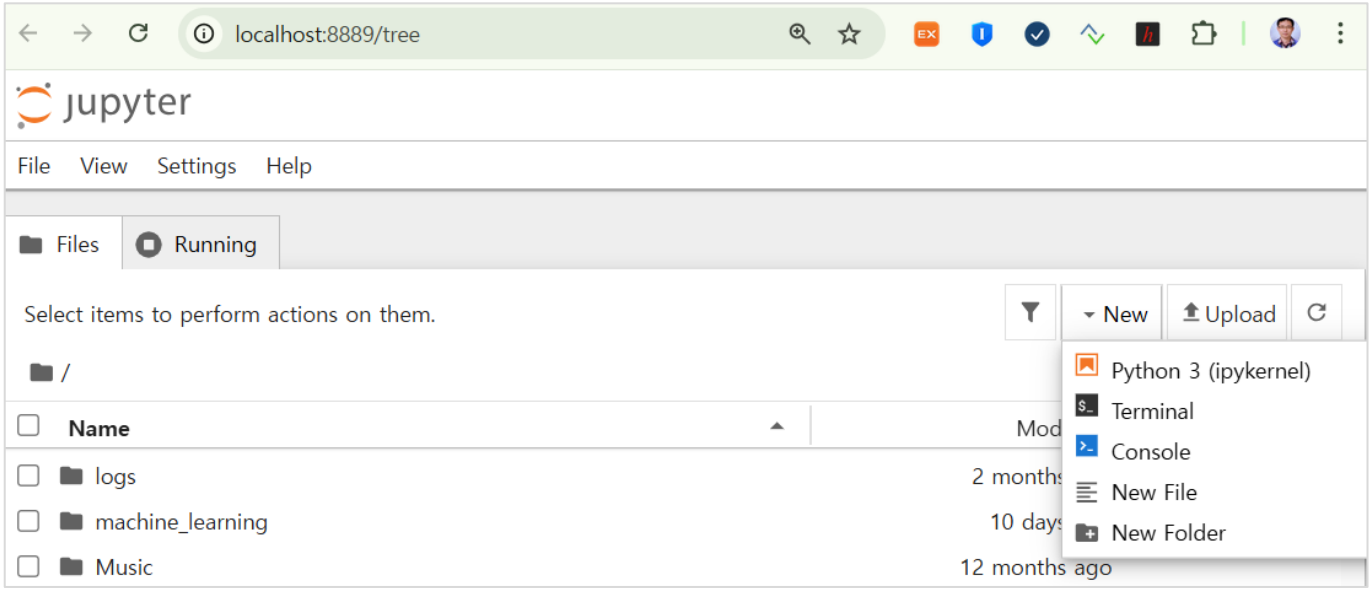


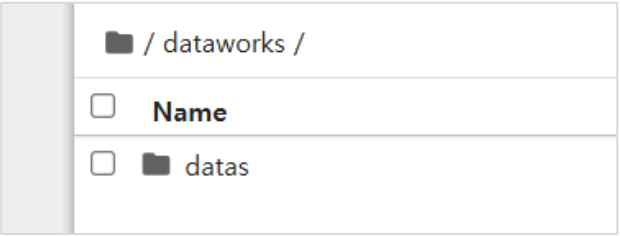


주피터 노트북(Jupyter Notebook)

❖ 작업 디렉터리 생성

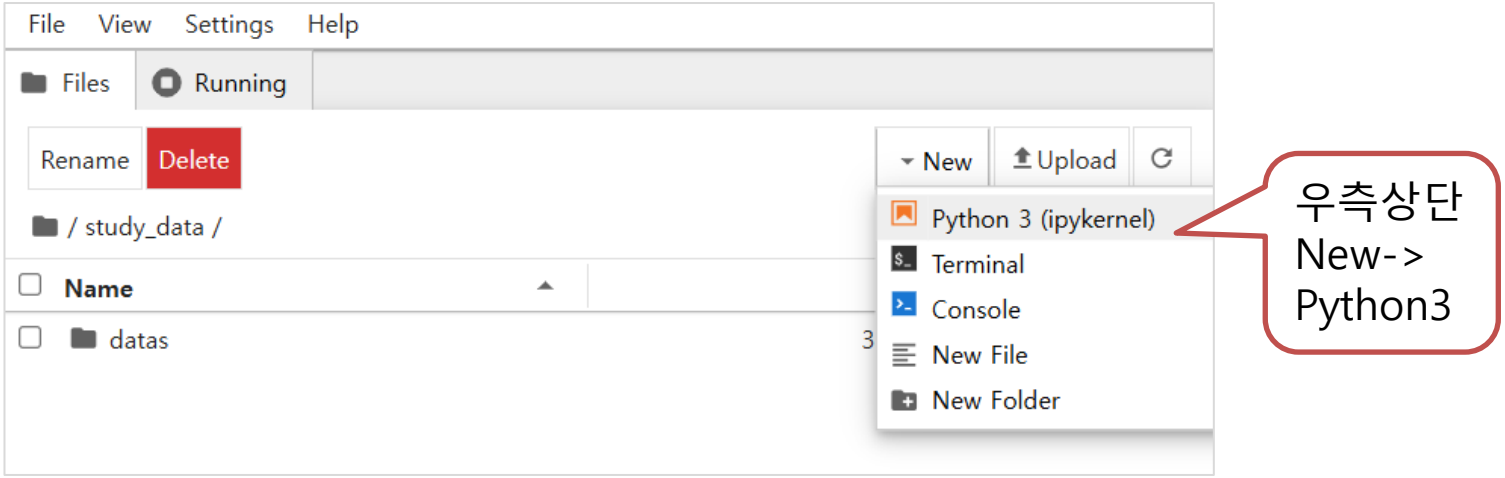
New > New Folder > dataworks 폴더 생성 > 하위에 datas 폴더 생성

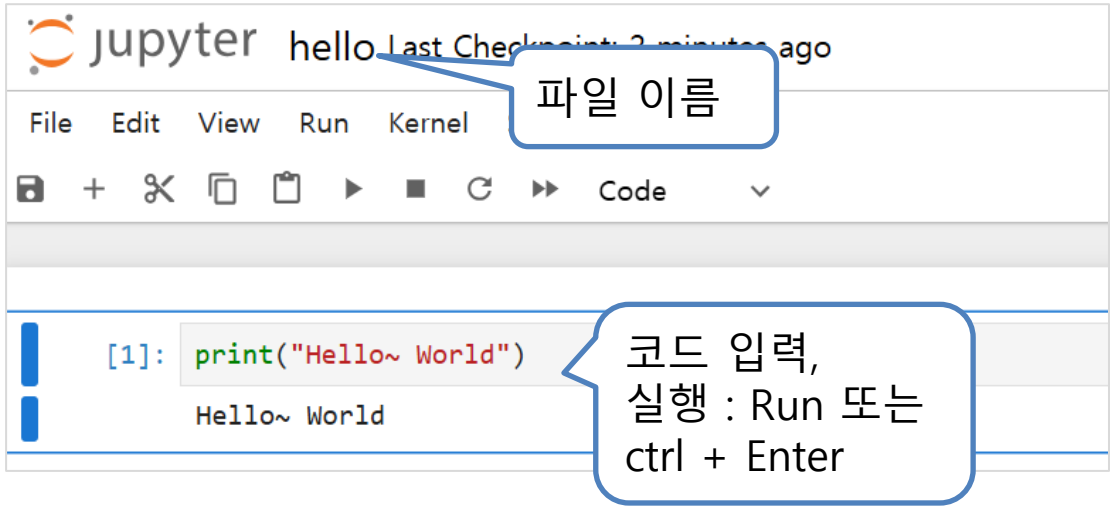




주피터 노트북(Jupyter Notebook)

❖ 파일 생성(.ipynb)





주피터 노트북(Jupyter Notebook)

❖ 코드 입, 출력

```
[1]: print("Hello~ World")

     Hello~ World

[2]: print("안녕~ 세계")

     안녕~ 세계

[7]: 10 + 20
     10 - 20

[7]: -10
```

```
[6]: n1 = 20
     n2 = 10

     print(n1 + n2)
     print(n1 - n2)
     print(n1 * n2)
     print(n1 / n2)
     print(n1 % n2)

     30
     10
     200
     2.0
     0
```

```
[10]: carts = ["계란", "라면", "콩나물"]
      print(carts)

      for cart in carts:
          print(cart)

      ['계란', '라면', '콩나물']
      계란
      라면
      콩나물
```

주피터 노트북(Jupyter Notebook)

❖ 세팅(Settings)

- 자동 완성 기능

  Home > Settings > Settings Editor > Enable autocompletion

- 줄번호, 글꼴 등

  Home > Settings > Notebook

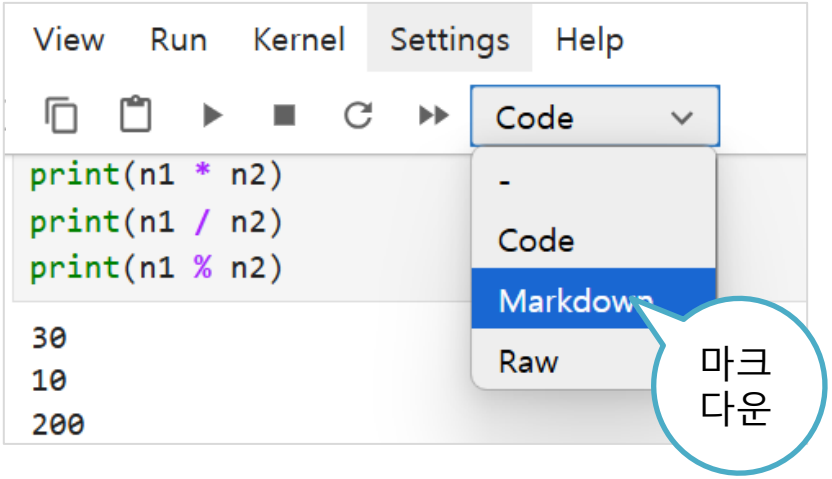
  - line numbers
  - Font Size : 20

주피터 노트북(Jupyter Notebook)

❖ Markdown

Markdown은 *설명·정리·문서화*를 담당하는 아주 중요한 기능이에요. 코드 옆에 **글, 표, 수식, 이미지**를 깔끔하게 적을 수 있어서 "노트 + 코드" 느낌으로 쓰기 딱 좋아요.

- 기호 종류
  - 제목: #의 개수가 클수록 작은 제목(## 텍스트)
  - 목록 – '*' 사용



주피터 노트북(Jupyter Notebook)

❖ Markdown 편집 및 실행 화면

```
### 함수 만들기
* 기능 - 절대값 계산
* 설명 - 음수는 양수로 양수는 양수로 반환
```

```
def my_abs(x):
    if x < 0:
        return -x
    else:
        return x

print(my_abs(-3))
```

```
3
```

<편집 화면>

Ctrl + Enter

함수 만들기

- 기능 - 절대값 계산
- 설명 - 음수는 양수로 양수는 양수로 반환

```
def my_abs(x):
    if x < 0:
        return -x
    else:
        return x

print(my_abs(-3))
```

```
3
```

<실행 화면>

데이터 분석

- **데이터 분석용 주요 라이브러리**

+ 데이터 분석을 위한 주요 라이브러리를 알려줘

라이브러리	주요 역할	핵심 기능
pandas	데이터 분석 핵심	DataFrame, CSV/Excel 처리, 필터링, 그룹화
numpy	수치계산 기반	다차원 배열, 수학/통계 연산, 고속 연산
matplotlib	데이터 시각화	선/막대/산점도 그래프

데이터 분석

- **판다스**
  - 데이터를 수집, 정리하는 데 최적화된 도구로 **표 형태(행·열)의 데이터**를 빠르고 편리하게 처리하기 위한 라이브러리입니다.
  - 가장 인기있는 프로그래밍 언어인 **파이썬(python)**을 기반으로 한다.
  - 시리즈(Series)와 데이터프레임(DataFrame)이라는 구조화된 데이터 형식을 제공한다.
  - 판다스 설치 - **아나콘다 플랫폼**에는 이미 설치되어 있습니다.

    (pip install pandas)

데이터 분석

- **판다스 자료구조**
  - **시리즈(Series)** – 데이터가 순차적으로 나열된 1차원 배열의 형태를 가지며, 인덱스(Index)는 데이터 값(value)과 일대일 대응이 된다.
  - **데이터프레임(DataFrame)** – 행(Row)과 열(Column)로 만들어지는 2차원 배열 구조로 표(Table) 형태이다.

	제품명
0	키보드
1	마우스
2	USB 메모리

인덱스 (index)

[시리즈]

	제품명	가격
0	키보드	20,000
1	마우스	32,000
2	USB 메모리	10,000

칼럼 (column)

[데이터프레임]

데이터 분석

- **Series(시리즈)**

import pandas as pd(별칭)

pd.Series(데이터, 인덱스)

속성	기능
index	- 시리즈의 인덱스 (생략하면 기본 0, 1, 2)
values	- 시리즈의 값(요소)
shape	- 시리즈의 크기 (요소의수,)

- **Series(시리즈)**

시리즈(Series) 만들기

```python
import pandas as pd

data = ['키보드', '마우스', 'USB 메모리']

s1 = pd.Series(data, index=[1, 2, 3])
print(s1)
print(type(s1))

# Series의 속성
print(s1.index)
print(s1.values)
print(s1.shape)

# 요소 선택 - 리스트 처럼 순서가 아닌 라벨 인덱스
print(s1[1])
print(s1[2])
```

```
1        키보드
2        마우스
3    USB 메모리
dtype: object
<class 'pandas.core.series.Series'>
Index([1, 2, 3], dtype='int64')
['키보드' '마우스' 'USB 메모리']
(3,)
키보드
마우스
```

데이터 분석

- **DataFrame(데이터프레임)**

```
import pandas as pd(별칭)

pd.DataFrame(데이터, 인덱스)
```

속성	기능
index	- 데이터프레임의 인덱스 (생략하면 기본 0, 1, 2)
columns	- 데이터프레임의 칼럼
shape	- 데이터프레임의 크기 (행, 열)

데이터 분석

칼럼(Column) 다루기

칼럼 추가 및 칼럼명 변경

```python
import pandas as pd

df = pd.DataFrame({
    "name": ['키보드', '마우스', 'USB 메모리'],
    "price": [20000, 32000, 11000]
})

# 칼럼 추가
df['spec'] = ['유선', '무선', '128GB']

df['할인가'] = df['price'] * 0.8
print(df)
print("----------------------------------------")

# 수정후 반드시 df에 다시 저장.
df = df.rename(columns={"name": "품명", "price":"가격", "spec":"제품상세"})
print(df)
```

```
      name  price   spec      할인가
0     키보드  20000    유선  16000.0
1     마우스  32000    무선  25600.0
2  USB 메모리  11000  128GB   8800.0
----------------------------------------
       품명     가격  제품상세      할인가
0     키보드  20000    유선  16000.0
1     마우스  32000    무선  25600.0
2  USB 메모리  11000  128GB   8800.0
```

데이터 분석

- **데이터 프레임을 CSV 파일로 변환**

CSV(Comma Seperated Value) 파일: 쉼표(,)로 데이터를 구분하여 저장하는 텍스트 파일 형식이다. 주로 표형태의 데이터를 저장하는 데 사용하며 엑셀에서 쉽고 열고 편집할 수 있음

☞ CSV 파일로 변환

df.to_csv("computer.csv", index=False)

☞ CSV 파일 읽기

pd.read_csv("computer.csv")

데이터 분석

- **데이터 프레임을 CSV 파일로 변환**

CSV 파일이란?

쉼표(,)로 데이터를 구분하여 저장하는 텍스트 파일 형식이다.

엑셀에서 .csv 형식으로 저장하고 불러올 수 있다.

```python
import pandas as pd

# csv 파일 만들기 - index 제외
df.to_csv("datas/computer.csv", index=False)

# csv 파일 읽기
print(pd.read_csv("datas/computer.csv"))
```

```
        품명     가격   제품상세      할인가
0      키보드  20000     유선  16000.0
1      마우스  32000     무선  25600.0
2  USB 메모리  11000  128GB   8800.0
```

/ study_data / datas /

Name
computer.csv

데이터 관리

- **행과 열 선택**

속성	범위	예시
loc[인덱스명, 칼럼명] (location)	끝 인덱스 포함	인덱스 - 라벨인덱스
iloc[인덱스번호, 칼럼번호] (integer location)	끝 인덱스 - 1	인덱스 – 순서(0부터 시작)

데이터 관리

- **행/열 선택 : loc[행이름, 열이름] 사용**

```python
# csv 파일 읽어서 df 객체에 저장
df = pd.read_csv("datas/computer.csv")

# 행 선택 - loc[행(행이름)]
print(df.loc[0:1])
print("========================================")

# 특정 요소 선택 - loc[행이름, 열(칼럼명)]
print(df.loc[0, "품명"])  #키보드
print(df.loc[1, "가격"])  #32000
print(df.loc[0:1, "품명"])
print(df.loc[0:1, ["품명", "가격"]])
```

```
    품명     가격 제품상세      할인가
0  키보드  20000   유선  16000.0
1  마우스  32000   무선  25600.0
========================================
키보드
32000
0    키보드
1    마우스
Name: 품명, dtype: object
    품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- **행/열 선택 : iloc[행번호, 열번호] 사용**

```
# 행 선택 - iloc[행(인덱스번호)]
print(df.iloc[0:2]) # 끝인덱스-1
print("========================================")

# 특정 요소 선택 - loc[행(인덱스번호), 열(인덱스번호)]
print(df.iloc[0, 0]) #키보드
print(df.iloc[1, 1]) #32000
print(df.iloc[0:2, 0])
print(df.iloc[0:2, 0:2])
```

```
    품명     가격 제품상세      할인가
0  키보드  20000   유선  16000.0
1  마우스  32000   무선  25600.0
========================================
키보드
32000
0    키보드
1    마우스
Name: 품명, dtype: object
    품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- 데이터 프레임 - 요소값 변경(업데이트)

df.loc[행이름, 열이름] = “변경할 값 "

df.iloc[행번호, 열번호] = “변경할 값 "

- 데이터 프레임 - 요소값 추가

df[열] = “추가할 칼럼“ => 열 추가

df.loc[행] = “추가할 값“ => 행 추가

데이터 관리

- **데이터 프레임 - 행/열 요소 변경 및 추가**

```python
# 요소값 변경 및 추가
df2 = df.copy() #원본 유지 복사
df2

# 품명 '마우스'를 '무선마우스'로 변경
df2.loc[1, "품명"] = "무선마우스"
# 무선 마우스의 가격을 35000으로 변경
df2.loc[1, "가격"] = 35000
# 무선 마우스의 할인가 수정
df2.loc[1, "할인가"] = df2.loc[1, "가격"] * 0.8
df2

# 행 추가
df2.loc[3] = ["키보드", 40000, "무선", 0.0]
df2.loc[3, "할인가"] = df2.loc[3, "가격"] * 0.8
df2
```

	품명	가격	제품상세	할인가
0	키보드	20000	유선	16000.0
1	무선마우스	35000	무선	28000.0
2	USB 메모리	11000	128GB	8800.0
3	키보드	40000	무선	32000.0

데이터 관리

- 데이터 프레임 - 행/열 삭제

df.drop(행, axis=0) => 행 삭제

df.drop(열, axis=1) => 열 삭제

```
# 행 삭제 - drop(행번호, axis=0)
# 열 삭제 - drop(칼럼명, axis=1)
df3 = df2.copy()
df3

# 3행 삭제
df3 = df3.drop(3, axis=0)
df3

# 열 삭제
# df3 = df3.drop("할인가", axis=1)
df3 = df3.drop(["제품상세", "할인가"], axis=1)
df3
```

	품명	가격
0	키보드	20000
1	무선마우스	35000
2	USB 메모리	11000

데이터 탐색

- **데이터 프레임을 CSV 파일로 변환**

```
# 데이터프레임 만들기
food = pd.DataFrame({
    "메뉴" : ["김밥", "비빔밥", "자장면", "우동", "김밥", "자장면", "김밥"],
    "가격" : [3000, 7000, 5500, 5500, 3500, 6000, 4000],
    "분류" : ["한식", "한식", "중식", "일식", "한식", "중식", "한식"]
})
# print(food)

# csv 파일로 변환
food.to_csv("food.csv", index=False)

# csv 파일 읽어오기
df = pd.read_csv("food.csv")
print(df)
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```
# 데이터 출력 - head(), tail() -> 기본 5개
print(df.head())
print(df.tail())

# 데이터 정보 - info()
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   메뉴      7 non-null      object
 1   가격      7 non-null      int64
 2   분류      7 non-null      object
dtypes: int64(1), object(2)
memory usage: 300.0+ bytes
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```python
# 고유한 값의 개수 - value_counts()
print(df["메뉴"].value_counts())
print(df["분류"].value_counts())

# 통계 요약 - describe()
# print(df.describe())
print(df.describe(include='number'))
print(df.describe(include='object'))
```

```
메뉴
김밥      3
자장면     2
비빔밥     1
우동      1
Name: count, dtype: int64
분류
한식     4
중식     2
일식     1
Name: count, dtype: int64
```

	가격
count	7.000000
mean	4928.571429
std	1455.694892
min	3000.000000
25%	3750.000000
50%	5500.000000
75%	5750.000000
max	7000.000000

데이터 탐색

- **조건 검색(필터링)**

비교 연산 기호	설명
>, >=	크다, 크거나 같다.
<, <=	작다, 작거나 같다
==	두 항이 같다.
!=	두 항이 같지 않다.

논리 연산	연산 기호	설명
교집합(AND)	&	두 조건이 모두 참일때 True 반환
합집합(OR)	|	두 조건중 하나 이상이 참일때 True 반환
부정(NOT)	~	조건이 참이면 False, 거짓이면 True를 반환

데이터 탐색

- **데이터 탐색 – 조건 검색(필터링)**

```
# 조건 검색(필터링)
# 메뉴가 '자장면'인 음식의 정보
result1 = (df["메뉴"] == '자장면')
print(df[result1])

# 메뉴가 '김밥'이 아닌 음식의 정보
result2 = (df["메뉴"] != '김밥')
# result2 = ~(df["메뉴"] == '김밥')
print(df[result2])

# 가격이 4000원 미만인 음식의 정보
result3 = (df["가격"] < 4000)
print(df[result3])

# 메뉴가 '자장면'이고, 가격이 6000원 이상인 음식의 정보
# result4 = (df["메뉴"] == '자장면') & (df["가격"] >= 6000)
result4 = (df["메뉴"] == '자장면') | (df["가격"] >= 6000)
print(df[result4])
```

데이터 탐색

● 데이터 탐색 - 정렬

정렬 분류	함수	파라미터
칼럼 기준	sort_values(칼럼, 파라미터)	오름차순: ascending=True 내림차순: ascending=False
인덱스 기준	sort_index(파라미터)	

```
    메뉴    가격   분류
1  비빔밥  7000  한식
5  자장면  6000  중식
2  자장면  5500  중식
3   우동   5500  일식
6   김밥   4000  한식
4   김밥   3500  한식
0   김밥   3000  한식
```

```
    메뉴    가격   분류
6   김밥   4000  한식
4   김밥   3500  한식
0   김밥   3000  한식
1  비빔밥  7000  한식
3   우동   5500  일식
5  자장면  6000  중식
2  자장면  5500  중식
```

데이터 탐색

- **데이터 탐색 - 정렬**

```
df = pd.read_csv("datas/food.csv")
# print(df)
# print("========================================")

# 가격을 내림차순으로 정렬하기
df.sort_values("가격", ascending=False, inplace=True)
print(df)
print("========================================")

# 메뉴를 오름차순으로 정렬, 메뉴가 같으면 가격을 내림차순 정렬
df.sort_values(["메뉴", "가격"], ascending=[True, False], inplace=True)
print(df)
print("========================================")

# 인덱스 기준 정렬(초기화)
df.sort_index(ascending=True)
print(df)
```

데이터 탐색

- **판다스의 통계, 수학 , 그룹핑 함수**

함수	기능
count() / sum() / mean()	개수 / 합계 / 평균
var() / std()	분산 / 표준편차
max() / min()	최대값 / 최소값
idxmax() / idxmin()	최대값 위치 / 최소값 위치
quantile(.25) / quantile(.75)	1사분위수 / 3사분위수
mode()	빈도값
round(수, 자리수)	반올림
groupby(칼럼명)	그룹화

데이터 탐색

- 판다스의 통계, 수학 함수

```
print("개수:", df["가격"].count())
print("합계:", df["가격"].sum())
print("평균:", round(df["가격"].mean(), 2))
print("분산:", round(df["가격"].var(), 2))
print("표준편차:", round(df["가격"].std(), 2))
print("최대값:", df["가격"].max())
print("최소값:", df["가격"].min())
print("최대값의 위치:", df["가격"].idxmax())
print("최소값의 위치:", df["가격"].idxmin())
```

데이터 탐색

- **판다스의 통계, 수학 함수**

```
# 사분위수 - quantile()
print("1사분위수:", df["가격"].quantile(.25))
print("2사분위수:", df["가격"].quantile(.50))
print("3사분위수:", df["가격"].quantile(.75))
print("4사분위수:", df["가격"].quantile(1.0))

# 조건 - 1사분위수 보다 작은 가격을 출력
result = df["가격"] < df["가격"].quantile(.25)
print(df[result])

# 최빈값 - mode()
print("최빈값:", df["가격"].mode()[0])
```

데이터 탐색

- **그룹핑 - groupby()**

```python
import pandas as pd

# "수량" 칼럼 추가
df["수량"] = [10, 10, 15, 5, 7, 7, 8]
df

# csv 파일로 변환
df.to_csv("./datas/food2.csv", index=False)

# csv 파일 읽기
food = pd.read_csv("./datas/food2.csv")
food
```

```
메뉴,가격,분류,수량
김밥,3000,한식,10
비빔밥,7000,한식,10
자장면,5500,중식,15
우동,5500,일식,5
김밥,3500,한식,7
자장면,6000,중식,7
김밥,4000,한식,8
```

데이터 탐색

- **그룹핑 - groupby()**

```
food = pd.read_csv("food2.csv")
# print(food)

# 그룹화 - groupby()
df["분류"].value_counts()

food.groupby("메뉴").mean(numeric_only=True)
food.groupby("분류").mean(numeric_only=True) #모든 수치 칼럼 평균

food.groupby("분류")["가격"].mean()  #분류별 가격의 평균
food.groupby("분류")["수량"].mean()  #분류별 수량의 평균
```

데이터 전처리

- **결측치 삭제 및 대체**

함수	설명
drop(행, axis=0) drop(열, axis=1)	NaN이 포함된 행 삭제 NaN이 포함된 열 삭제
dropna()	NaN이 포함된 모든 행 삭제
fillna()	- 수치형: 평균(mean()), 중간값(median()) - 문자형: 빈도값(mode())
isnull() isnull().sum()	- 결측치 확인(True / False로 반환) - 결측치 개수(True(1))

데이터 전처리

- **결측치 대체**

```
data = {
    "메뉴": ["김밥", "비빔밥", "자장면", "우동", None, "자장면", "김밥"],
    "가격": [3000, 7000, 5500, None, 3500, 6000, 4000],
    "분류": ["한식", "한식", "중식", "일식", "한식", "중식", "한식"]
}

food = pd.DataFrame(data)
food
food.info()

# 결측치 확인 - isnull()
print(food.isnull())
print(food.isnull().sum()) #True(1) / False(0)
```

```
메뉴    1
가격    1
분류    0
```

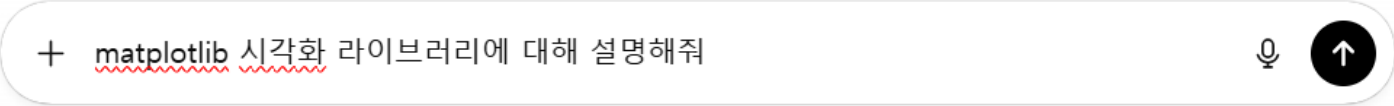
데이터 전처리

- **결측치 대체**

```
# 수치형 - 중간값으로 대체
median = food["가격"].median()
median

# fillna()
food["가격"] = food["가격"].fillna(median)
food

# 문자형 - 최빈값
frequency = food["메뉴"].mode()[0]
frequency
food["메뉴"] = food["메뉴"].fillna(frequency)
food

print(food.isnull().sum())
```

데이터 시각화

- **matplotlib 라이브러리**



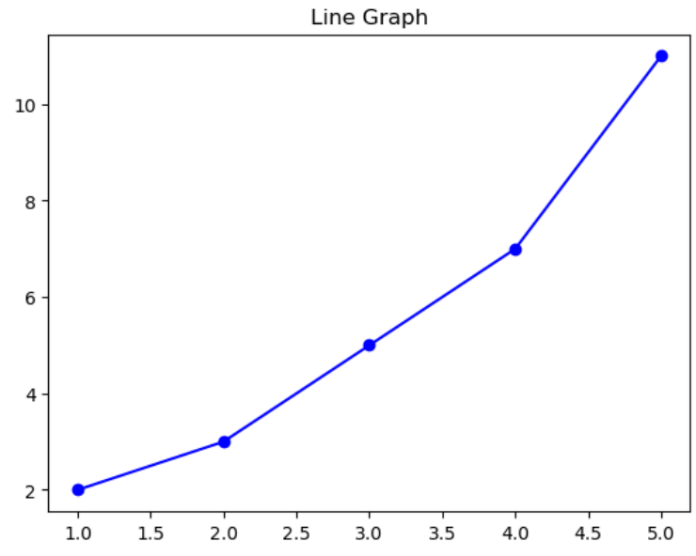
- 데이터 분석, 과학 시각화, 보고서 작성 등 다양한 분야에서 그래프를 그릴 때 사용됩니다.
- 주요 그래프 종류

종류	함수명	예시
선 그래프	plot()	plt.plot(x, y)
막대 그래프(수직/수직)	bar() / barh()	plt.bar(x, y) / plt.barh(x, y)
산점도	scatter()	plt.scatter(x, y)
히스토그램	hist()	plt.hist(data)
파이차트	pie()	plt.pie(data)

70

데이터 시각화

- 선 그래프



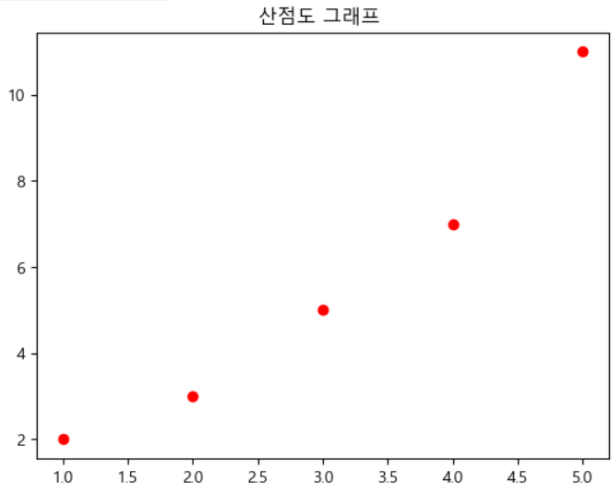

```python
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# 선 그래프 그리기
plt.plot(x, y, label='Prime numbers', color='blue', marker='o')
plt.title('Line Graph') #제목
plt.legend() #범례

# 그래프 출력
plt.show()
```


71

데이터 시각화

- 산점도 그래프

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]  #소수

plt.rc('font', family='Malgun Gothic')
plt.title('산점도 그래프 ')
plt.scatter(x, y, color='red')
plt.show()
```

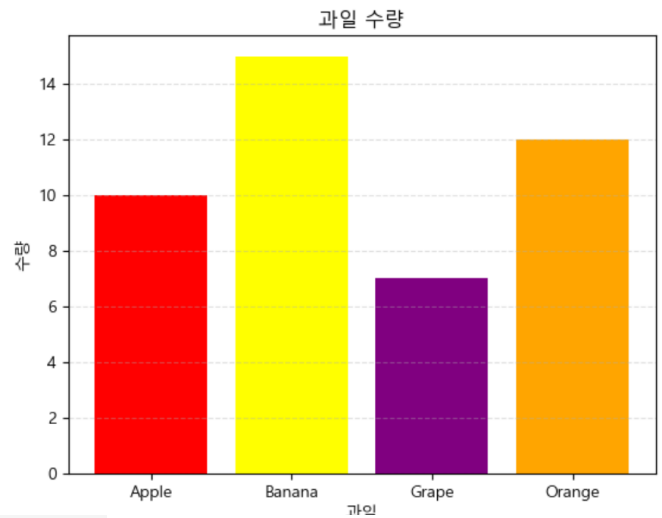


72

데이터 시각화

- 막대 그래프

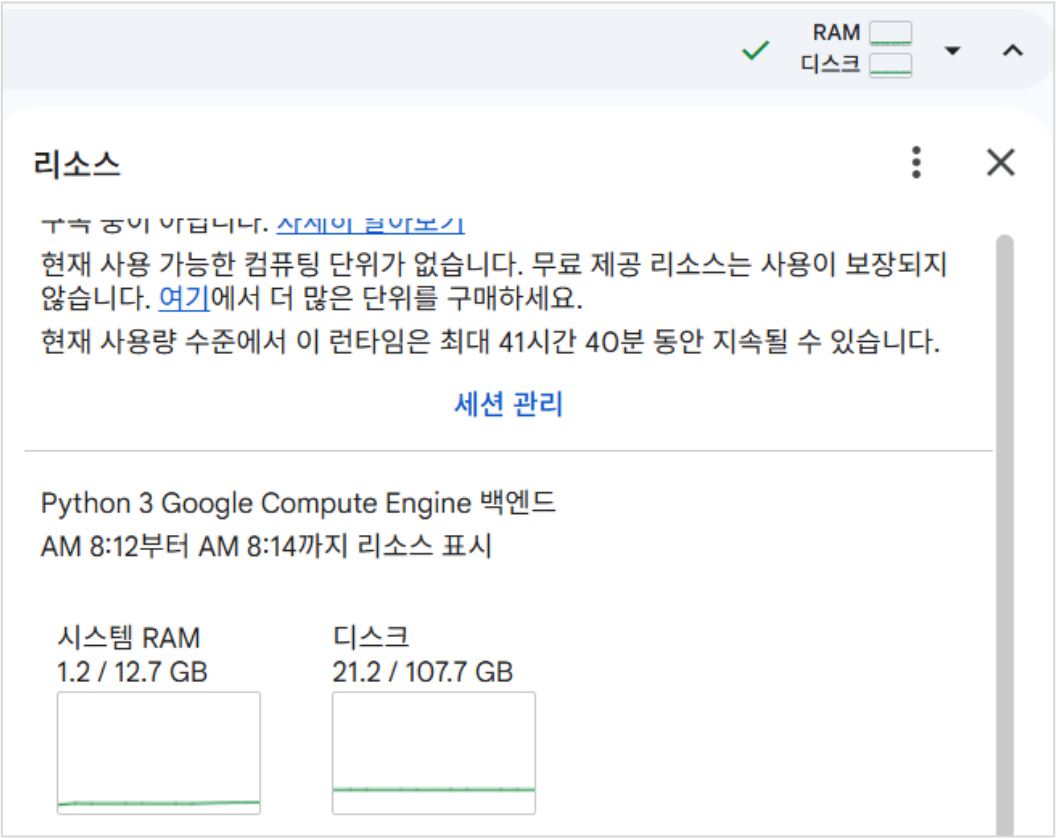
```python
import matplotlib.pyplot as plt

# 데이터 준비
fruits = ['Apple', 'Banana', 'Grape', 'Orange']
counts = [10, 15, 7, 12]
color = ['red', 'yellow', 'purple', 'Orange']

# 한글 깨짐 방지 - Batang, Gulim 사용
plt.rc('font', family="Malgun Gothic")

# 막대 그래프
plt.bar(fruits, counts, color=color)
plt.title('과일 수량')
plt.xlabel('과일') # x축 제목
plt.ylabel('수량') # y축 제목

# y축 grid 추가
# plt.grid()
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```



73

구글 코렙

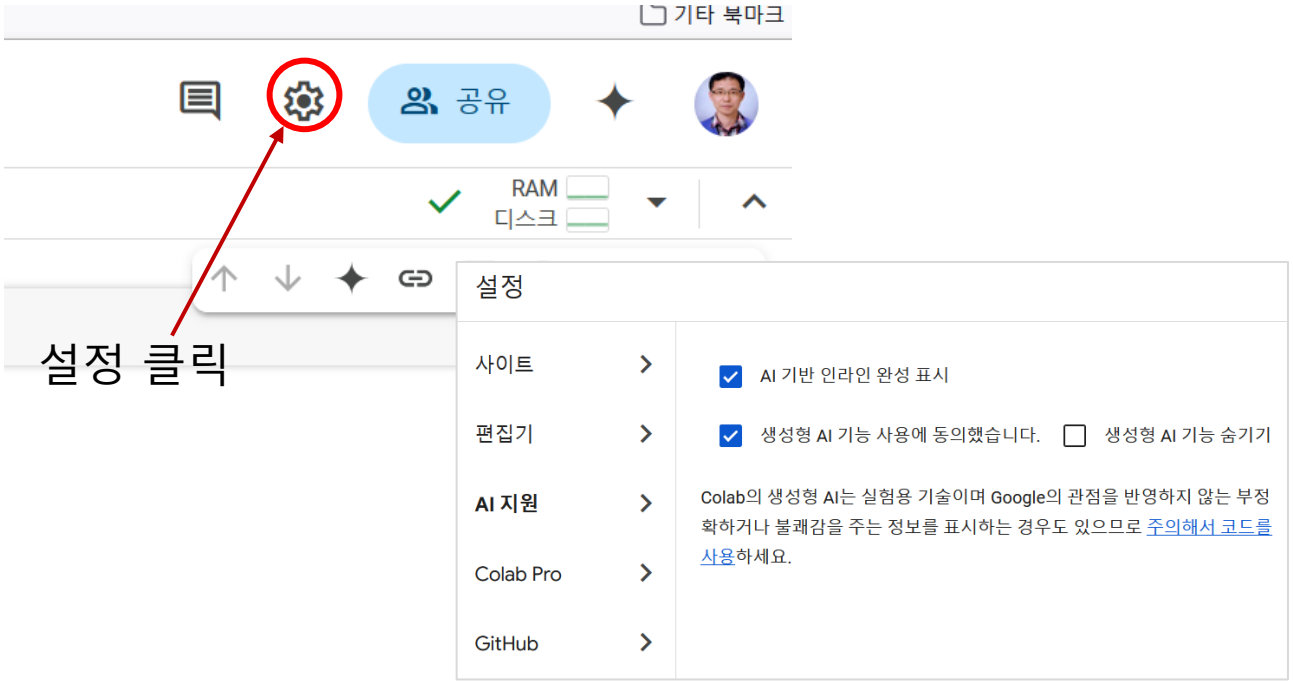
❖ 리소스 사용 – 클라우드 컴퓨터 사용



구글 코렙

❖ 구글 코랩(Colaboratory)

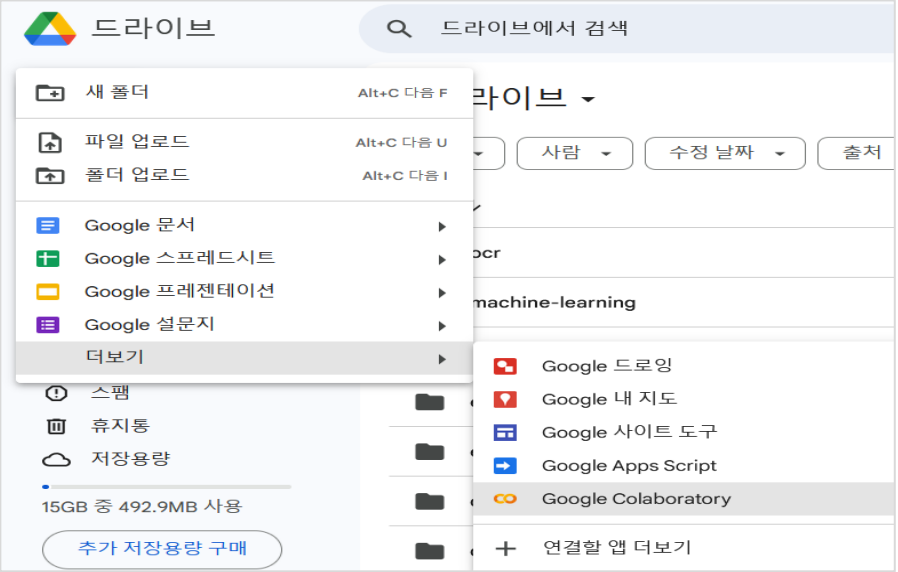
AI(Gemini) 지원 : 설정 > AI 지원 > 동의



구글 코렙

❖ 새 노트 만들기

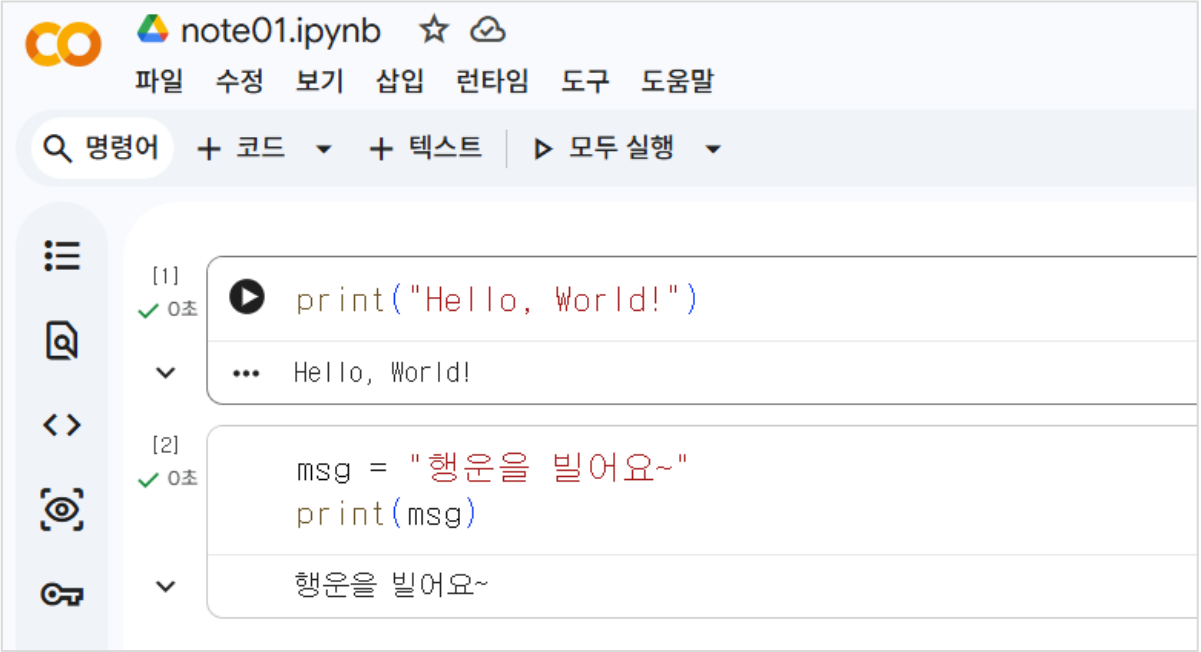
- 드라이브 > 홈
- + 신규 > 더보기 > Google Colaboratory



구글 코렙

❖ 새 노트 만들기

- 파일이름 > note01.ipynb



구글 코렙

❖ 새 노트 만들기

- 파일이름 > note01.ipynb

```python
import pandas as pd

df = pd.DataFrame({
    "name": ['마우스', '모니터'],
    "price": [20000, 80000]
})

df
```

	name	price
0	마우스	20000
1	모니터	80000

다음 단계: df 변수로 코드 생성 | New interactive sheet

구글 코렙

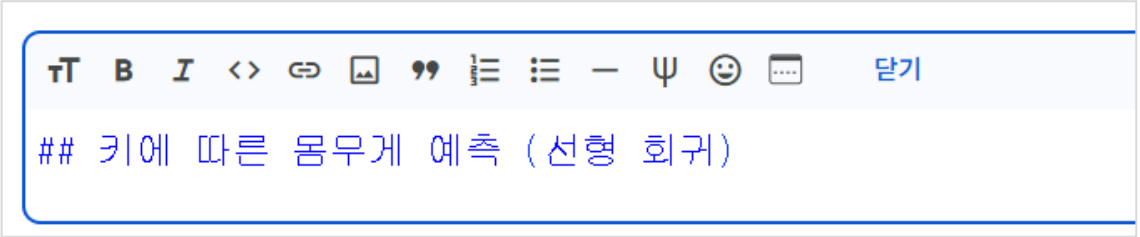
❖ 새 폴더 만들기

- 드라이브 > 내 드라이브
- + 신규 > 새폴더> 머신러닝

구글 코렙

❖ 텍스트 사용하기

- 파일 이름 : 선형회귀.ipynb
- 텍스트(버튼) > 문서의 제목을 만들때 사용



<편집화면>

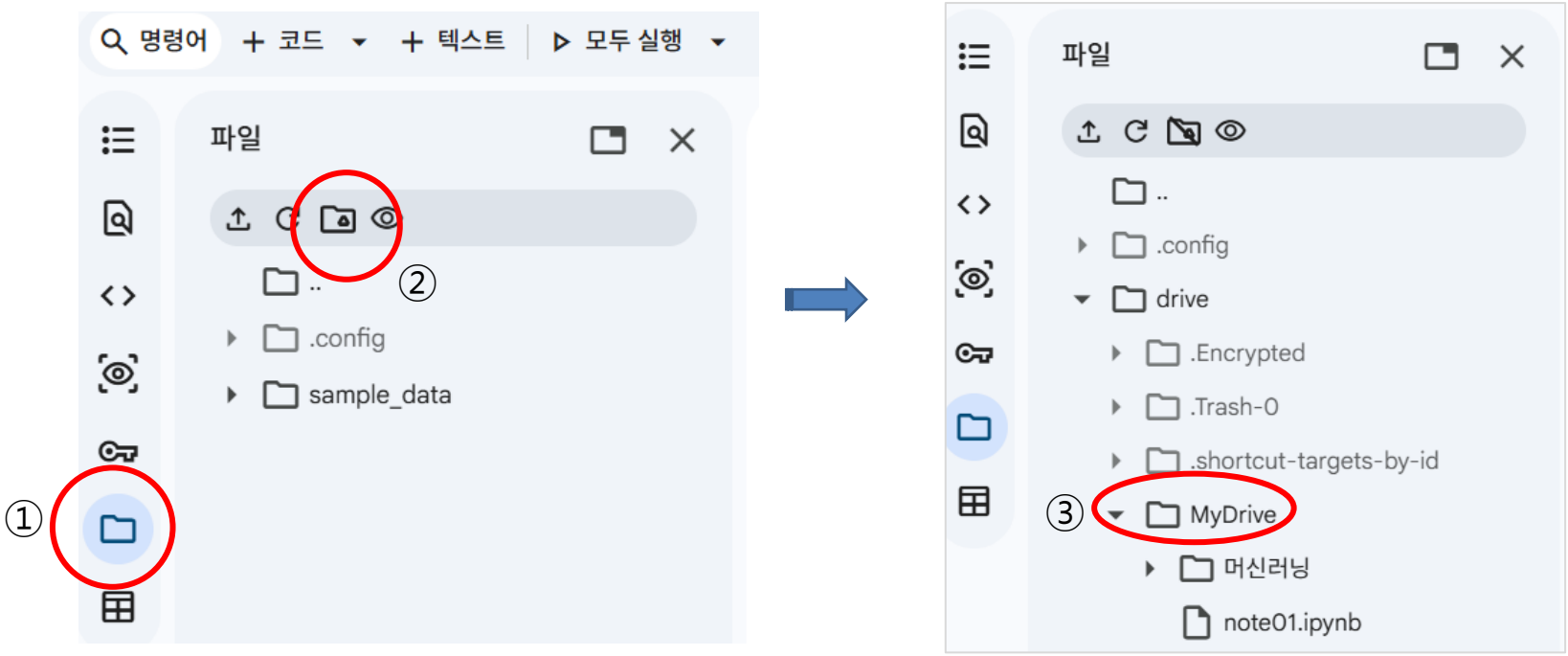
shift + enter

키에 따른 몸무게 예측 (선형 회귀)

<출력화면>

❖ 드라이브 마운트

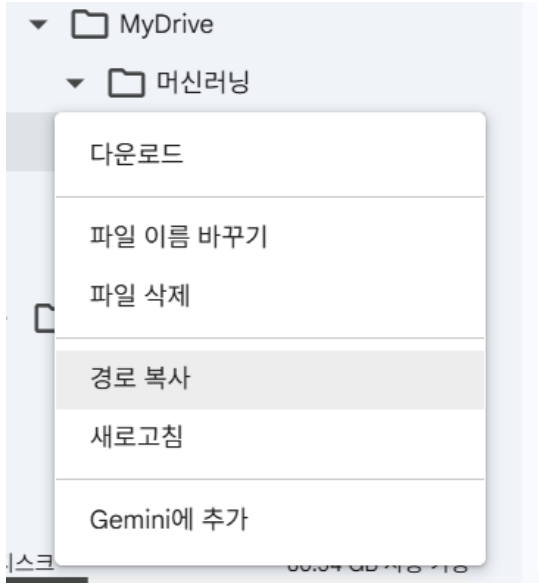
- 폴더 > 드라이브 마운트 > 드라이브 연결

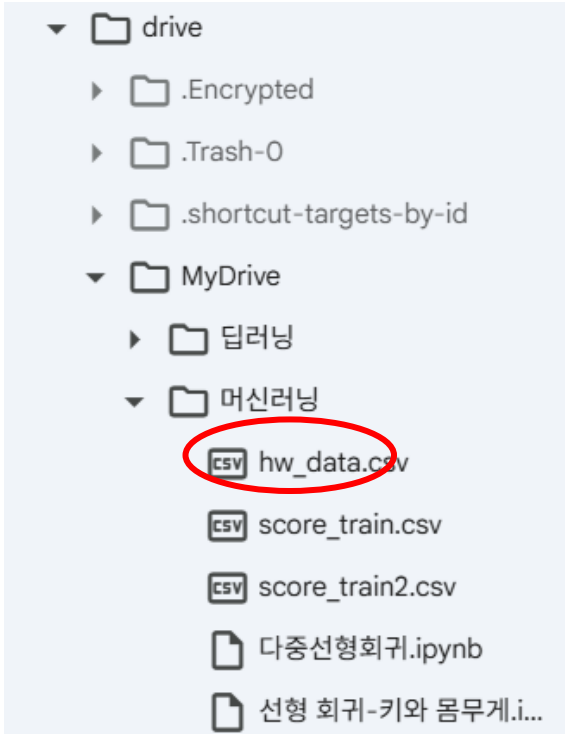


구글 코렙

❖ 훈련 데이터 업로드

- (...) 메뉴 > 업로드 > data_hw.csv





데이터 시각화

- **seaborn 라이브러리**
  - Python의 데이터 시각화 라이브러리로, 통계적 그래프를 쉽게 그릴 수 있도록 matplotlib을 기반으로 만들어졌다
  - pandas의 DataFrame과 잘 통합되어 있어, 데이터를 시각적으로 분석하고자 할 때 매우 유용하다

- **설치 방법**

  pip install seaborn

83

타이타닉호 사례 분석

- 타이타닉호 데이터 준비

```
import seaborn as sns

df = sns.load_dataset('titanic')
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no

84

타이타닉호 사례 분석

- 타이타닉호 데이터 정보(결측치 확인)

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	891 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	embark_town	891 non-null	object
12	alive	891 non-null	object
13	alone	891 non-null	bool

85

타이타닉호 사례 분석

- **타이타닉호 데이터 분석 및 처리**

```python
# 결측치 처리
# df.isnull().sum()
print(df.isna().sum())

# age, embarked, embark_town 칼럼 - 결측치 대체
# deck 칼럼 - 결측치 삭제
df2 = df.copy() #원본 복사

# age - 평균값 계산
mean_age = df2['age'].mean()
print(mean_age) #29.699

# 평균값으로 채우기
df2['age'] = df2['age'].fillna(mean_age)

# embarked - 최빈값 계산
most_freq = df2['embarked'].mode()[0]
print(most_freq) #S

df2['embarked'] = df2['embarked'].fillna(most_freq)
print(df2.isna().sum())
```

```
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
embarked         0
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

타이타닉호 사례 분석

- 타이타닉호 데이터 분석 및 처리

```
# embark_town - 최빈값 이용
print(df2['embark_town'][825:830])

most_freq = df2['embark_town'].mode()[0]
print(most_freq) #Southampton

# 최빈값으로 채우기
df['embark_town'] = df['embark_town'].fillna(most_freq)
print(df['embark_town'][825:830])

# deck열 삭제
df2 = df2.drop('deck', axis=1)
df2.info()
print(df2.isna().sum())
df2.columns # 칼럼 확인
```

```
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
embark_town    0
alive          0
alone          0
dtype: int64
```


87

타이타닉호 사례 분석

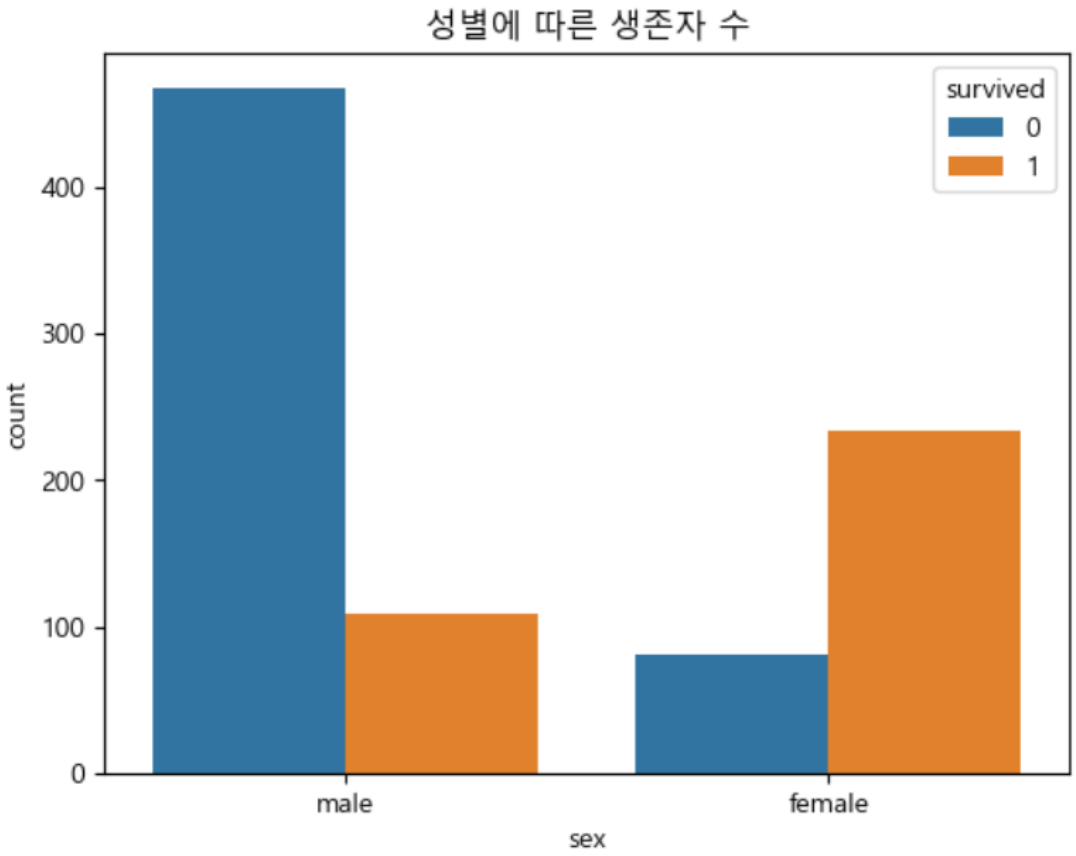
- **타이타닉호 - 성별에 따른 생존자 수 그래프**

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 예제 데이터셋 로드
df = sns.load_dataset('titanic')

# 데이터 확인
print(df.head())

# 성별에 따른 생존자 수 시각화 (막대그래프)
sns.countplot(data=df, x='sex', hue='survived')
plt.title('성별에 따른 생존자 수')
plt.show()
```


88

타이타닉호 사례 분석

- 타이타닉호 - 성별에 따른 생존자 수 그래프



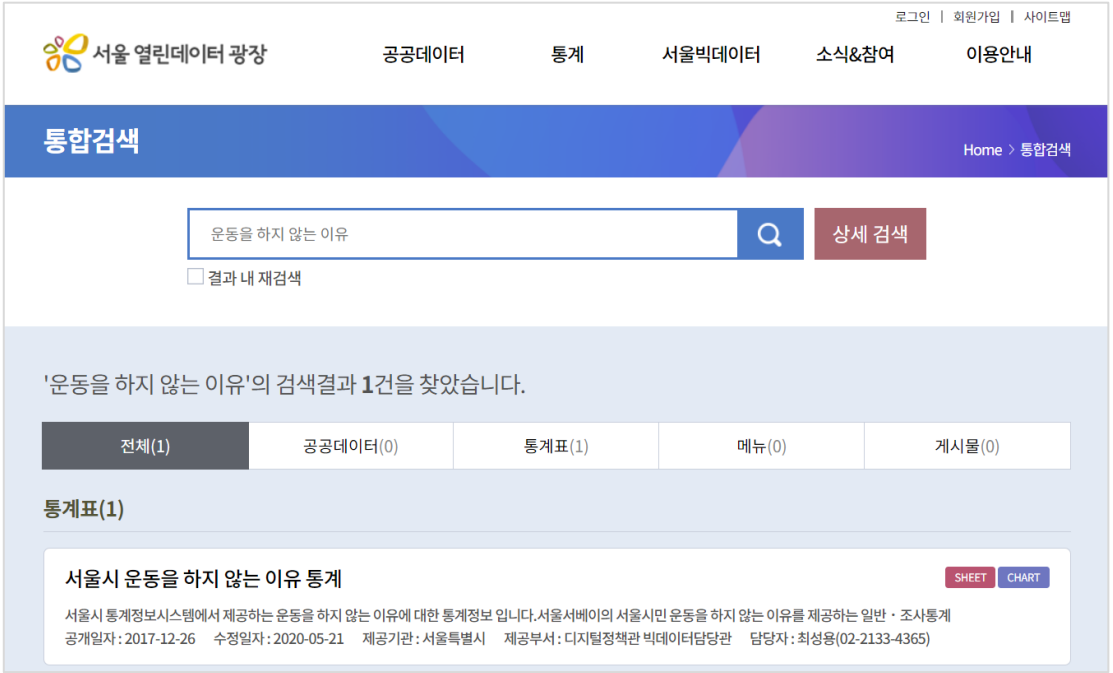
89

공공데이터 분석 및 시각화

- 자료 수집
  - 열린 데이터 광장(data.seoul.go.kr)

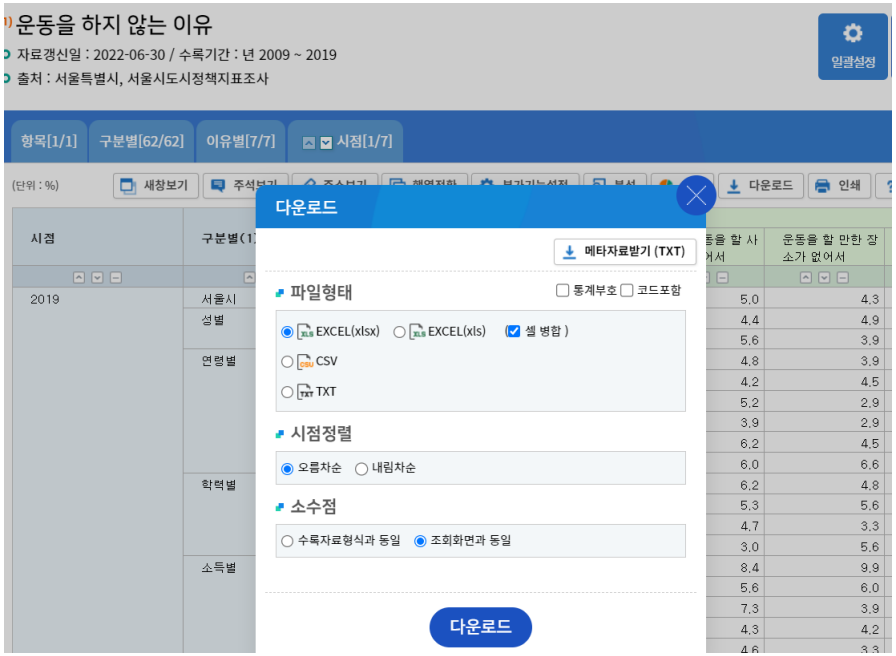
서울시 운동을 하지 않는 이유 통계

- 자료 수집
  - 운동을 하지 않는 이유 검색



생활 운동을 하지 않는 이유 통계

- 자료 수집
  - excel 파일 다운로드



- 판다스로 엑셀 파일 읽어 오기
  - **pd.read_excel**(파일명)

```
import pandas as pd
import warnings
warnings.simplefilter('ignore')

# 제목행의 1번 인덱스만 출력(0번은 숨김)
not_exercise = pd.read_excel("./datas/exercise.xlsx", header=1)
not_exercise.head()
```

	시점	구분별 (1)	구분별 (2)	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서	운동을 할 만한 장소가 없어서
0	2019.0	서울시	소계	46.8	5.0	4.3
1	NaN	성별	남자	52.4	4.4	4.9
2	NaN	NaN	여자	42.5	5.6	3.9
3	NaN	연령별	10대	55.3	4.8	3.9
4	NaN	NaN	20대	46.0	4.2	4.5

사용자 운동을 하지 않는 이유 통계

- 데이터 전처리
  - **'시점' 칼럼 삭제**

```
# '시점' 칼럼 확인
print(not_exercise.columns[0])

# 칼럼 삭제 - 객체.drop()
not_exercise = not_exercise.drop('시점', axis=1)
not_exercise
```

시점

	구분별(1)	구분별(2)	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서
0	서울시	소계	46.8	5.0
1	성별	남자	52.4	4.4

- 데이터 전처리
  - **칼럼명 변경**

```
# 칼럼명 변경 - 객체.rename(), inplace=True(즉시 저장 및 실행)
not_exercise.rename(columns={'구분별(1)':'대분류', '구분별(2)':'분류'}, inplace=True)
not_exercise
```

	대분류	분류	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서	운동을
0	서울시	소계	46.8	5.0	
1	성별	남자	52.4	4.4	
2	NaN	여자	42.5	5.6	
3	연령별	10대	55.3	4.8	
4	NaN	20대	46.0	4.2	

- 데이터 전처리
  - **행 삭제**

45	NaN	영등포구	50.2
46	NaN	동작구	39.1
47	NaN	관악구	44.4
48	NaN	서초구	49.9
49	NaN	강남구	40.7
50	NaN	송파구	47.6
51	NaN	강동구	47.0

```
# 행 삭제 - 객체.drop(index=range(시작행, 끝행-1))
not_exercise.drop(index=range(22, 52), inplace=True)
not_exercise
```

- 데이터 전처리
  - **행 수정**

```
# 2행의 NaN을 '성별'로 변경 - '대분류' 칼럼의 2행을 수정
not_exercise.loc[2, '대분류'] = '성별'
not_exercise
```

	대분류	분류	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서
0	서울시	소계	46.8	5.0
1	성별	남자	52.4	4.4
2	성별	여자	42.5	5.6

- 데이터 전처리
  - **조건 검색**

```
# 대분류가 성별인 항목만 검색

not_exercise['대분류'] == '성별'
not_exercise[not_exercise['대분류'] == '성별']
```

	대분류	분류	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서
1	성별	남자	52.4	4.4
2	성별	여자	42.5	5.6

- 데이터 전처리
  - **깊은 복사 & 얕은 복사**

```
# 원본 유지 - 깊은 복사

not_ex_gender = not_exercise[not_exercise['대분류'] == '성별'].copy()  #깊은 복사
not_ex_gender
```

얕은 복사, 깊은 복사

```
# 얕은 복사
a = [1, 2, 3, 4]
a
```

```
[1, 2, 3, 4]
```

```
b = a
b
```

```
[1, 2, 3, 4]
```

```
b[1] = 10
b
```

```
[1, 10, 3, 4]
```

```
a   # b리스트를 변경하면 a(원본)도 똑같이 변경됨
```

```
[1, 10, 3, 4]
```

```
# 깊은 복사
a = [1, 2, 3, 4]
a
```

```
[1, 2, 3, 4]
```

```
b = a.copy()
b
```

```
[1, 2, 3, 4]
```

```
b[1] = 10
b
```

```
[1, 10, 3, 4]
```

```
a   # b리스트를 변경해도 a는 원본을 유지함
```

```
[1, 2, 3, 4]
```

- 데이터 전처리
  - **칼럼 삭제**

```
# 대분류 칼럼 삭제
# not_ex_gender.drop("대분류", axis=1, inplace=True)
not_ex_gender.drop(columns='대분류', inplace=True)
not_ex_gender
```

	분류	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서
1	남자	52.4	4.4
2	여자	42.5	5.6

- 데이터 전처리
  - **인덱스 설정**

```python
# index를 '분류'로 세팅
not_ex_gender.set_index('분류', inplace=True)
not_ex_gender
```

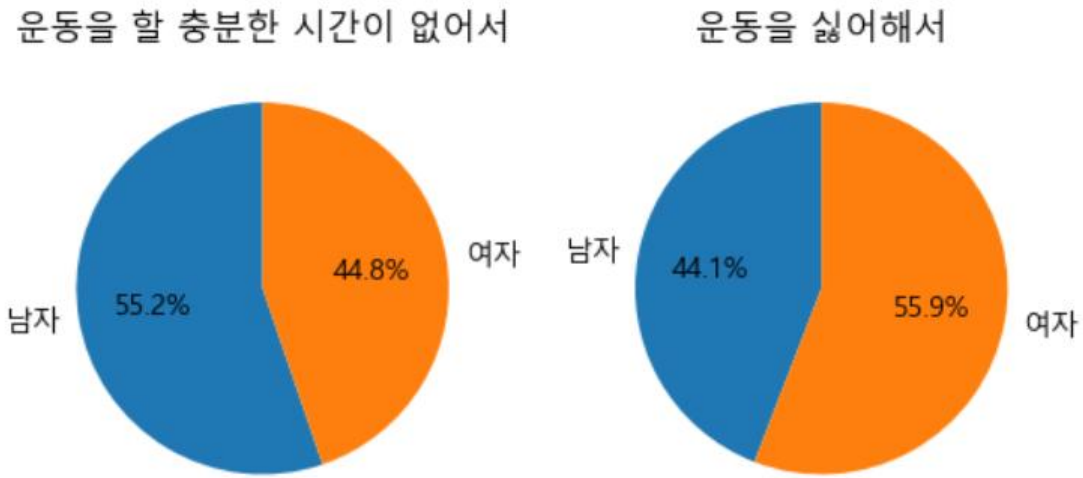
분류	운동을 할 충분한 시간이 없어서	함께 운동을 할 사람이 없어서
남자	52.4	4.4
여자	42.5	5.6

- 시각화
  - **파이 차트 그리기**

fig(figure) : 그래프를 그릴 공간

ax(axis) : 그 공간중 사용할 부분

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 2)  # subplots() - 작은 그래프(1행 2열)
plt.rc('font', family='Malgun Gothic')

not_ex_gender['운동을 할 충분한 시간이 없어서'].plot.pie(ax=ax[0],
                autopct='%.1f%%', startangle=90)
ax[0].set_title('운동을 할 충분한 시간이 없어서')
ax[0].set_ylabel('')

not_ex_gender['운동을 싫어해서'].plot.pie(ax=ax[1],
                autopct='%.1f%%', startangle=90)
ax[1].set_title('운동을 싫어해서')
ax[1].set_ylabel('')
```

- 시각화
  - **파이 차트 그리기**

```
Text(0, 0.5, '')
```